



IMPLEMENTATION GUIDE SDK ANDROID AUTTAR

VERSION 1.2

Summary

INTRODUCTION	4
1 DOCUMENTATION USAGE	4
2 EXAMPLE	4
3 SETUP SDK – AUTARPAYMENTSSDK	5
4 LOGS	6
5 SDK INITIALIZATION	6
6 CONFIGURATIONS	6
7 AUTHENTICATION	7
8 CONNECTING TO UI	7
9 GUIDELINE FOR CALL-BACK SCREENS	7
1.1 SHOWQRCODE	7
1.2 HIDEQRCODE	8
1.3 ASKGENERICCAPTURE	8
1.4 ASKFORVALUE	10
1.5 MESSAGEDISPLAY	11
1.6 ASKFORDATE	11
1.7 CLEANDISPLAY	12
1.8 ASKFORCONFIRM	12
1.9 ASKSELECT	13
1.10 SHOWINSERTCARDMESSAGE	14
1.11 SHOWPIN	15
10 STARTING TRANSACTION	16
11 TRANSACTION RESULT	17
12 PAYMENT SPLIT	17
13 MULTI TRANSACTIONS	18
14 MULTIEC	19
15 CONFIGURATIONS	20
15.1 SUBMERCHANT	20
16 SUPPORTED DEVICES	23
17 ERROR CODES	23
18 SDK ERROR CODES	24

Introduction

The Auttar Android SDK solution is a solution that inherits all the features of the standard TEF solution, which, following the Auttar complete solution guidelines, provides a solution for a Mobile payment method for the Android environment. The solution has full support for the EMV library, which guarantees the capture of Credit and Debit operations, in the chip, magnetic and contact modalities in a secure way, in addition to payment via QR Code and PIX and Tap on Phone.

1 Documentation usage

Three folders are sent next to this document, the “documentation” folder contains the SDK documentation itself.

It is possible to use it minimally by opening index.html, but to view it completely, the ideal is to execute the following commands within the directory:

```
npm i
npm run start
```

It will launch a web page, which can be accessed at <http://127.0.0.1:8080/> (localhost). As seen below:



On this page you will find how to use the SDK itself, which is located in the “repository” folder.

2 Example

The “example” folder has a complete application with an example of using the SDK, including credit and debit transactions, and examples of how to call screens, it is a good starting point to start the implementation.

3 Setup SDK – AuttarPaymentsSDK

To use AuttarPaymentsSDK you need to add its dependencies to your project and an additional dependency matching the desired target: TapOnP, Pinpad or POS.

```
//Obrigatório para todos os targets
implementation("br.com.auttar.mobile:payment-sdk-debug:$auttarSdkVersion")

//POS
implementation("br.com.auttar.mobile:payment-sdk-bc-pos-debug:$auttarSdkVersion")

//Pinpad
implementation("br.com.auttar.mobile:payment-sdk-bc-pinpad-debug:$auttarSdkVersion")

//TOP
implementation("br.com.auttar.mobile:payment-sdk-top-debug:$auttarSdkVersion")
```

The integration app also is required to have a desugaring solution like bellow :

```
android{
    compileOptions {
        coreLibraryDesugaringEnabled true
    }
}

dependencies {
    coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:2.0.4")
}
```

If you application is targeting POS its also necessary to add the following tag to its application manifest.

```
<activity
    android:name="br.com.avatek.bc.pin.PinFragment"
    android:exported="false"
    tools:node="merge" />
```

4 Logs

By default writing logs into files is disabled, however it can be enabled by using `IAuttarConfiguration.saveSupportFiles`.

```
val downloadDir = Environment
    .getExternalStoragePublicDirectory(Environment.DIRECTORY_DOWNLOADS)
    .path
Auttar.getConfigSDK().saveSupportFiles(downloadDir) {
    Toast.makeText(
        this,
        "$it\n\nContendo dados para suporte foi salvo em $downloadDir",
        Toast.LENGTH_LONG
    ).show()
}
```

5 SDK Initialization

To begin using the SDK, the first required step is to initialize it by passing a reference to your Android Application instance, specifically the Auttar object.

It is recommended to perform this initialization inside the `onCreate()` method of your main Application class, ensuring the SDK is properly set up before any other components are loaded.

```
class SDKApplication : Application() {

    override fun onCreate() {
        super.onCreate()
        Auttar.initHomolog(this)
    }
}
```

The Auttar object has three init methods available depending on the environment in which you intend to perform transactions:

- **Auttar.initHomolog()** : Use this method for testing purposes in a homologation environment.
- **Auttar.initProduction()** : Use this method for real payment transactions in the production environment.
- **Auttar.initStandAlone()** : Use this method to manually configure a CTF Server.

6 Configurations

The Auttar SDK exposes its configuration options through the method `Auttar.getConfigSDK()`, which returns an instance of `IAuttarConfigSDK`.

The first configuration that must be set is related to scanning permissions, which are essential for enabling device-level scanning features used during transactions.

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Auttar.getConfigSDK().scanPermissions(this)
}
```

7 Authentication

The IAuttarSDK has necessary methods for authentication which must be called before starting transactions.

```
Auttar.getSDK().authenticationMonoEC(
    ParameterLoginIntegrationMonoEC(
        user = "username",
        password = "password",
        cnpj = "08697957000140",
        storeCode = null
    ), object : LoginResultListener {
        override fun onResult(result: LoginResultInterface) {
            val isLoginSucessfull = result.code == 0
            val message = result.message
        }
    })
```

8 Connecting to UI

During the transaction flow, the SDK is responsible for initiating user interactions such as requesting a date input, a monetary value, or prompting the user to swipe a card. To support these interactions, the integrating application must implement and provide an instance of **ClientUserInterface**. This interface allows the SDK to delegate UI-related tasks to the host application, ensuring the user interface can be updated appropriately based on the SDK's requirements, **make sure** to call **IAuttarSDK.connectUI** before starting any transaction.

```
val clientUserInterface : ClientUserInterface
Auttar.getSDK().connectUI(clientUserInterface)
```

9 Guideline for call-back screens

Both in the example and in the documentation, we see the need to implement the **ClientUserInterface** interface, which is nothing more than the interface for screen call-backs, it has methods that must call different screens in the application.

This guideline provides examples of screens for each of these calls, which are examples and do not necessarily need to be developed as faithful copies of these screens. Use your visual pattern to create your own canvases, using this guideline as inspiration.

1.1 showQrCode

This method asks for a screen displaying a Qr code to be displayed, both for Qr Code and Pix transactions. Example:



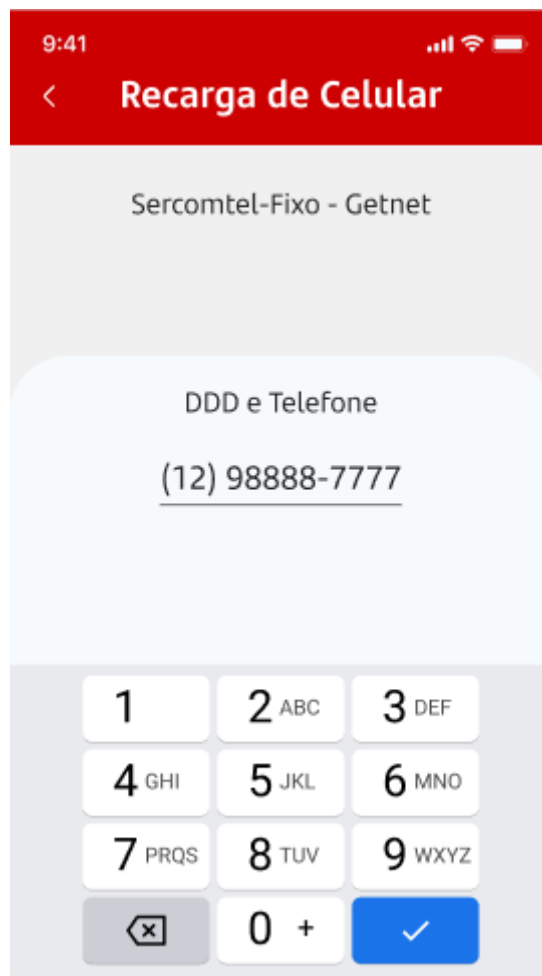
In the example above, the qr code is sent via `qrCodeImage`, being a bitmap, the amount and payment type are sent via `transactionValue` and `transactionType` respectively and `pinCode` is an alternative code to type (sent only in QR code transactions, it is not sent in case of PIX).

1.2 `hideQrCode`

This method should clear the Qr code screen and, if necessary, delete the generated image file (optional).

1.3 `askGenericCapture`

A screen with generic String capture should open, masks are sent to be used together, an example is the screen for capturing CPF and telephone number, example:



The screenshot shows a mobile application interface for 'Recarga de Celular' (Mobile Recharge). The top status bar displays the time 9:41, signal strength, Wi-Fi, and battery icons. The app's title bar is red with a back arrow and the text 'Recarga de Celular'. Below this, the provider is identified as 'Sercomtel-Fixo - Getnet'. A light blue rounded rectangle contains the label 'DDD e Telefone' and the entered number '(12) 98888-7777'. At the bottom is a numeric keypad with digits 1-9, 0, a backspace key, and a confirmation key with a checkmark.

9:41

< Recarga de Celular

Sercomtel-Fixo - Getnet

DDD e Telefone

(12) 98888-7777

1 2 ABC 3 DEF

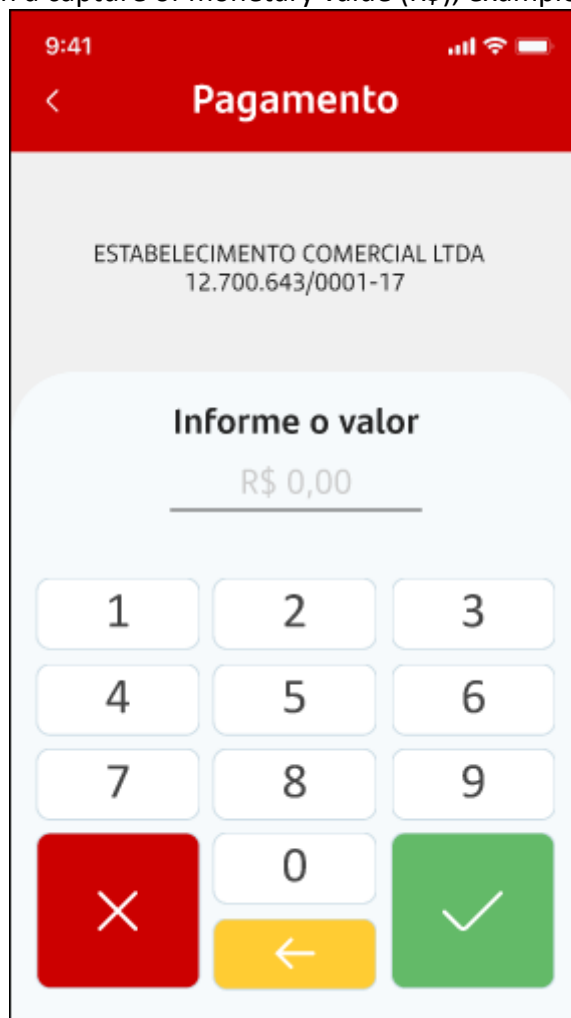
4 GHI 5 JKL 6 MNO

7 PRQS 8 TUV 9 WXYZ

⌫ 0 + ✓

1.4 askForValue

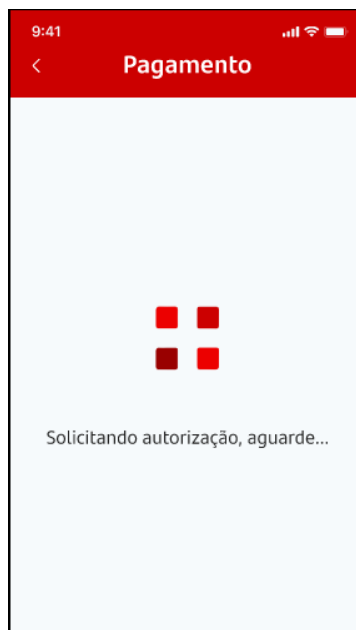
A screen should open with a capture of monetary value (R\$), example:



Where in this case the function's display parameter sent:
"ESTABELECIMENTO COMERCIAL LTDA 12.700.643/0001-17"

1.5 messageDisplay

Screen that should display a text message coming from the message parameter, as in the image below:



1.6 askForDate

Screen for capturing dates or times (see the mask parameter to identify the type), example:



1.7 cleanDisplay



It should clear the current screen, we recommend leaving a wait screen at these times, for example:

In the example below we leave a loading animation, we recommend this to give a more pleasant experience to the user, but this is optional.

1.8 askForConfirm

Displays a confirmation screen with the display message, example:



1.9 askSelect

It should display a list, where the user can choose one of the items, for example:



1.10 showInsertCardMessage

It must display a screen for reading the card, on this screen it is possible for the display to return messages such as: "swipe the card", "approach, insert or swipe the card", "insert or swipe the card", or "Approach the card ":



Feel free to include animations or illustrations to make this screen more pleasant for the user.

1.11 showPin

Password screen, this screen should display while waiting for the user to enter the password

Note: When using a POS or Tap On Phone, it is recommended to leave the message at the top of the screen, as the POS virtual keyboard will open, occupying around 40% of the screen at the bottom.

It is not necessary to develop the keyboard, it will be displayed as the example below by the SDK, or it will be requested in the Pinpad Bluetooth (if you are using one).

Example:



10 Starting Transaction

The `IAuttar.paymentTransaction` methods is used to start a transaction based on the type of Data passed.

```
Auttar.getSDK().paymentTransaction(
    CreditData(
        BigDecimal(10.00),
        TypeCreditTransaction.CREDITO_GENERICO
    )
)
```

The `paymentTransaction` method expects a `DefaultData<AuttarTypeTransaction>`, where the `AuttarTypeTransaction` is a sealed interface provided by the sdk with the code for the supported TEF operation.

AuttarTypeTransaction	Descrição
TypeCreditTransaction	112 – A Vista 113 – Crédito Parcelado Lojista (sem juros) 114 – Crédito Parcelado Administradora (com juros) 119 – Pré-Autorização 120 – A Vista Digitado 121 – Crédito Digitado Parcelado Lojista (sem juros) 122 – Crédito Digitado Parcelado Administradora (com juros) 132 – Captura Pré-autorização 167 – Captura Pré-autorização Parcelado Lojista (sem juros) 168 – Captura Pré-autorização Parcelado Administradora (com juros) 223 – Crédito genérico 412 – Alteração Pre Autorização
TypeDebitTransaction	101 – Débito 103 – Débito pré-datado 106 – Voucher 224 – Débito genérico 339 – Pagamento de Carnê
TypeCancellationTransaction	128 – Cancelamento (Crédito, Débito, QR Code e etc.)
TypeFinalizeTransaction	6 – Confirmação 7 – Desfazimento
TypeQueryTransaction	118 – Consulta financiamento (cartão Private Label)
TypeInitializeTransaction	225 – Inicio de dia 400 – Carga de tabelas forçada 415 – Teste de comunicação
TypeMultiEcTransaction	329 – Consulta estabelecimentos MultiEC

TypeRechargePhoneTransaction	174 – Compra crédito digital 283 – Consulta telefone
TypeQRCodeTransaction	420 – Pagamento com QR
TypePixTransaction	422 – Pagamento via PIX 423 – Consulta o status de uma transação PIX 431 – Devolução via Pix
TypeInvoiceTransaction	348 – Consulta Fatura Cartão Detalhada 183 – Pagamento Fatura Cartão

11 Transaction Result

Once a transaction is initiated, the SDK will automatically request the necessary data provided that a successful connection to the UI has been established.

The result of the transaction will be emitted through a flow, which can be collected and handled as shown in the example below:

```
Auttar.getSDK().transactionStateFlow.collect { transactionState:
AuttarTransactionState ->
    if (transactionState is AuttarTransactionState.Done) {
        val tefResult: TefResultInterface = transactionState.result
    }
}
```

12 Payment Split

CreditData and DebitData both have a public property called splitPaymentIntegration that can be configured to enable payment split.

```
Auttar.getSDK().paymentTransaction(
    CreditData(
        BigDecimal(0),
        TypeCreditTransaction.CREDITO_GENERICO
    ).apply {
        with(splitPaymentIntegration) {
            setSellerId("SellerID")
            setOrderId("OrderId")
            addProduct(
                code = "productCode",
                amount = 10,
                valorUnitario = BigDecimal(10.00),
                descricao = "ProductDescription",
                idSubSeller = "SubsellerId",
                valorJuros = null,
                valorPorcentagem = BigDecimal(5.00),
                valorTaxa = BigDecimal(1.00)
            )
        }
    }
)
```

13 Multi Transactions

In use cases that require multiple TEF operations, such as when a customer pays using two different cards, you must initiate a multi-transaction session by calling `IAuttarSDK.multiTransactionStarted()` before processing the individual transactions, this . After completing all TEF operations, call `IAuttarSDK.multiTransactionFinished()` to retrieve the results of each transaction, and then finalize the entire process by invoking `IAuttarSDK.finalizeTransaction()`.

In use cases that require multiple TEF operations, such as when a customer pays using two different cards, you must initiate a multi-transaction session by calling `IAuttarSDK.multiTransactionStarted()` before processing the individual transactions. This call disables automatic confirmation of each transaction at `DefaultData`, meaning the application is responsible for manually finalizing them. After completing all TEF operations, call `IAuttarSDK.multiTransactionFinished()` to retrieve the results of each transaction, and then explicitly finalize the entire process by invoking `IAuttarSDK.finalizeTransaction()`.

```
Auttar.getSDK().multiTransactionStarted()
...

//As many transactions as necessary
Auttar.getSDK().paymentTransaction()

...
Auttar.getSDK().multiTransactionFinished(object : FinalizeTransactionListener {
    override fun onSuccess(data: Array<FinalizeDataInterface>) {
        if (data.isNotEmpty()) {
            data.forEach { finalizeData ->
                val tefResult = finalizeData.tefResult
                auttarSdk.finalizeTransaction(finalizeData)
            }
        }
    }

    override fun onError(errorCode: String, message: String, action: String) {
        //Handle errors
    }
})
```

14 MultiEC

The SDK can be configured to process multiple establishments in the same terminal. The first change necessary to enable this is to authenticate using the `IAuttarSDK.authenticationMultiEC` :

```
Auttar.getSDK().authenticationMultiEC(ParameterLoginIntegrationMultiEC(
    user = "username",
    password = "password",
    groupEC = "Group EC",
    codeActive = "codeActive"
), object : LoginResultListener {
    override fun onResult(result: LoginResultInterface) {
        val isLoginSucessfull = result.code == 0
        val message = result.message
    }
})
```

Then it is necessary to obtain the list of registered ECs by making the following administrative transaction:

```
Auttar.getSDK().administrativeTransaction(
    MultiECQueryData(BigDecimal(0),
    TypeMultiECTransaction.CONSULTA_ESTAB_MULTI_EC)
)
```

The transaction above will update the `IAuttarSdk.transactionStateFlow`, and if successful , will return a list of available ECs at `TefResult` like example aboce

```
Auttar.getSDK().transactionStateFlow.collect { transactionState ->
    if (transactionState is AuttarTransactionState.Done) {
        transactionState.result.auttarMerchantMultiEC?.forEach { identifier ->
            val multiECIdentifier = identifier
        }
    }
}
```

Then you need to pass the `multiECIdentifier` to the `DefaultData` in transaction like the exemple bellow for `CreditData` :

```
auttarSdk.paymentTransaction(
    CreditData(
        BigDecimal(0),
        TypeCreditTransaction.CREDITO_GENERICO
    ).apply { this.identifierMultiEC = multiECIdentifier }
)
```

15 Configurations

Some configuration data is accessible through the IUserConfiguration interface, accessible through the userConfiguration property of the IAuttarConfigSDK facade contained in the Auttar object.

```
val configSDK: IAuttarConfigSDK = Auttar.getConfigSDK()
```

The data available for consultation is related to the registered terminal, as in the example below we have a CNPJ consultation linked to previously carried out authentication:

```
val configSDK = Auttar.getConfigSDK()
val CNPJ = configSDK.userConfiguration.CNPJ.value
```

15.1 Submerchant

It is possible to configure the SDK to use a submerchant for the transactions processed by the SDK.

The submerchant data is set through the config façade shown above using the method:

```
/**
 * Set the data for sub merchant transactions.
 * @param subMerchant Data class with the sub merchant information.
 * @return Task result with code = 0 in case of success, otherwise
 * code = -1
 */
suspend fun setSubMerchant(subMerchant: SubMerchant) :
SubMerchantResult
```

The data class SubMerchant should hold all the information needed as described in the table below:

Parameter	Type	Max. Size	Details
id	String	15	Alphanumeric submerchant ID. Example: 1248ADC
city	String	13	Submerchant city, only letters allowed (a-z and A-Z). Example: SAO PAULO
state	String	2	Submerchant state, only letters allowed (a-z and A-Z) Example: SP
zipCode	String	8	Submerchant zip code, numbers only. Example: 90123456

cpfCnpj	String	11/14	Submerchant CPF or CNPJ, numbers only. Example: 12345678901234
address	String	40	Submerchant address, alphanumeric string with space and some special characters allowed (% \$, . / & () + = < > - *) Example: AV NACOES UNIDAS, 1245
razaoSocial	String	99	Submerchant razao social, alphanumeric string with space and some special characters allowed (% \$, . / & () + = < > - *) Exemplo: GETNET ADQUIRENCIA E SERVICOS PARA MEIOS DE PAGAMENTO S.A
nomeFantasia	String	99	Submerchant fantasy name, alphanumeric string with space and some special characters allowed (% \$, . / & () + = < > - *) Example: Loja Santo Antônio da Pat
phone	String	11	Submerchant phone, numbers only. Example : 11987654321
url	String	255	Submerchant URL, mandatory only for e-commerce. Example: https://www.example.com.br OPTIONAL
idTransacao	String	40	Tag for partner internal control. Example: 47144560905_717e756e666d3e777833 OPTIONAL
idPagamento	String	20	Tag for partner internal control. Example: 000087806827933 OPTIONAL

After setting the submerchant the SubmerchantResult object will be returned with the operation result, in case of error the message field will contain a string describing the problem.

```
data class SubMerchantResult(
    var code: Int,
    var message: String? = null
)

enum class SubMerchantResultCodeEnum(val codeResult: Int) {
    INVALID_FIELD(-1),
    VALID(0)
}
```

IMPORTANT: The SDK will hold the last submerchant set and will automatically use it for every transaction executed. Setting a new one will replace the previous one.

You can check the submerchant information in use by the SDK through the config facade using the method below:

```
/**
 * Function to return the registered sub merchant
 * @return Returns the registered object SubMerchant or null if none
 is registered.
 */
suspend fun getSubMerchant() : SubMerchant?
```

To remove the submerchant and use the default transaction method you can use the following method from the same facade:

```
/**
 * Remove the stored sub merchant data
 */
suspend fun clearSubMerchant()
```

16 Supported Devices

- PIN PAD bluetooth:

Terminal	Fabricante
Link 2500	Ingênico
MP5	Gertec
D150	PAX
D200	PAX
ICMP	Ingênico
ME30S	Newland

- POS Android:

Terminal	Fabricante
APOS A8	Ingênico
DX8000	Ingênico
GPOS700	Gertec
N910	Newland
SUNMI	P2
SUNMI	P3

17 Error Codes

Error codes are formatted in a specific order to provide information about the source of the error and if is expected or not. Take the example bellow

UC20:5331

	U	C	20	5331
<u>Description</u>	If an error code starts if <u>U</u> it means it is not mapped by the SDK and it is and <u>unexpected error.</u>	This <u>field</u> represents the <u>source of error.</u>	Error <u>code</u> returned by CTF , BC or <u>Attestation</u>	Error <u>code</u> for client <u>or</u> <u>network.</u>
<u>Type</u>	<u>Char</u>	<u>Char</u>	<u>Int</u>	<u>String</u>
<u>Required</u>	<u>False</u>	<u>True</u>	<u>True</u>	<u>False</u>

Source Codes

<u>Char</u>	<u>Source</u>	<u>Public Enum</u>	<u>Details</u>
<u>A</u>	<u>Attestation Errors</u>	<u>CodeErrorAttestationEnum</u>	<u>Only for TOP</u>
<u>B</u>	<u>BC</u>	<u>CodeErrorBCEnum</u>	<u>Follows Pinpad ABECS error codes.</u>
<u>C</u>	<u>CTF</u>	<u>CodeErrorClientEnum</u>	
<u>N</u>	<u>Network</u>	<u>CodeErrorConnectionEnum</u>	
<u>S</u>	<u>SDK</u>	<u>CodeErrorSDKEnum</u>	<u>Error generated by the SDK itself</u>

18 SDK Error Codes

Código	Nome	Descrição
-1	INPUT_TRANSACTION	Invalid input passes as transaction parameters
-2	PROCESS_CLIENT	Unexpected error during payment flow
-3	DB_TRANSACTION_NO_DATA	to finalize a transaction when no records exist for the day
-4	DB_TRANSACTION_NOT_PENDING	Attempt to finalize a transaction that is not pending
-5	DB_TEF_RESULT_NO_DATA	Failed to find Tef Result for Multi Transaction finalization
-6	MULTI_TRANSACTION_RUNNING	Multi Transaction in progress
-7	MULTI_TRANSACTION_NO_DATA	Error finalizing Multi Transaction
-8	STANDALONE_NOT_CONFIGURED	SDK initialized incorrectly
-9	HOMOLOG_PROD_NOT_CONFIGURED	SDK initialized incorrectly
-10	NFC_COMPATIBILITY_ERROR	NFC functionality not recognized on the device

-11	NFC_DISABLE	NFC functionality disabled
-12	TOP_DND_UNAUTHORIZED	Android "Do Not Disturb" permission not granted
-14	PIN_PAD_BLUETOOTH_DISABLE D	Bluetooth Pinpad disabled
-15	TOKEN_EXPIRED_ERROR	App Token expired
-16	LOCATION_PERMISSION_DENIE D	Location permissions not granted
-17	LOCATION_DISABLE	Location disabled on Android device
-18	DB_TEF_RESULT_NSU_NOT_FO UND	Transaction record not found for the provided NSU
-19	TIME_OUT_WAIT_KEYS	Keys not returned to the device during the transaction
-61	KEYSTORE_NOT_VALID	SoftTRSM library failed to create/use the local Keystore for attestation
-62	DEVICE_NOT_SAFE	SoftTRSM library reported the device is not secure for unknown reasons
-63	UNKNOWN_ATT_ERROR	Unmapped attestation failure
502	BAD_GATEWAY	A&M servers returned 502 for Attestation, Info, and Keys operations
- 2024	CLIENT_ERROR_ON_TOP	Internal use
3000	CANCEL_USER_PASSWORD	Transaction canceled by user during PIN entry
3001	CANCEL_USER_BACKGROUND	Transaction canceled by user during the transaction
2000	GENERIC	Generic attestation error
2005	TAMPER_CAMERA	Device camera usage was detected during the transaction.

2016	TAMPER_SENSORS_FAILED	Failure in using sensors to generate random keys for the transaction
2004	TAMPER_SETTINGS	Sensitive Android settings detected during the transaction
0010	INTERNET_ERROR	Failed to communicate with A&M servers due to network issues